SIMATS ENGINEERING

Department of Information Technology

COMPILER DESIGN — UNIT I

ASSIGNMENT 1 — CO1

Development of a Domain-Specific Compiler Front-End using LEX for Federated Cyber Threat Detection in Security Operations Centers

Total Marks: 50 | Duration: 1 Week

Student detail	Entry	Student detail	Entry
Student Name	BALAJI ROHITH MV	Register Number	192521269
Department / Section	B.Tech IT	Academic Year / Semester	2026
Faculty Name	Dr.Deva Priya	Date of Submission	04.11.2026

Important source and evidence note

The supplied 17-page DOCX contains the assignment brief, rubric, mark sheet, submission checklist, and summary claims about a prior submission. It does not contain the referenced full answers, Appendix A/B source files, raw scanner logs, Table 9.1, Table 10.1 or the actual performance graphs. This document therefore answers the tasks from the supplied brief and established compiler/LEX principles. It does not represent unperformed compilation or benchmark runs as completed facts.

The supplied mark sheet records 49/50 and reports 0 of 5,000 new-token literals recognised and 13,040 spurious errors for the original design. These are repeated only as figures claimed by the source mark sheet; the underlying test data is not present and cannot be independently verified.

CONTENTS

● Language Processors
● Phases of a Compiler
● Grouping of Phases
● Compiler Construction Tools
● Role of the Lexical Analyzer
● Specification of Tokens
● Recognition of Tokens
● LEX Specification
● Implementation and Testing
● Scanner Performance
● Final Design Recommendation ●
Re-evaluation and Adaptation ●
References and evidence checklist

TASK 1 — LANGUAGE PROCESSORS

ThreatRule is a domain-specific language used by branch Security Operations Centers (SOCs) to express detection and response rules. The central SOC must validate shared rule files consistently before accepting them. A compiler is the most suitable primary processor because it checks complete rule structure, enforces semantic constraints and translates accepted rules into a common intermediate representation or executable form before deployment.

Processor	**Operation**	**Advantages**	**Suitability**
Compiler	Translates source into target code before execution.	Full lexical, syntax and semantic checks; optimisation; repeatable deployment.	Best fit for central validation and rule deployment.

Interpreter	Reads and executes source statements at runtime.	Interactive testing and rapid development.	Useful for debugging, but less suitable as the only central acceptance gate.
Assembler	Converts assembly mnemonics into machine code.	Useful for low-level target code or compiler output.	ThreatRule is a high-level DSL, not assembly.
Hybrid compiler / VM	Compiles to bytecode or IR executed/JIT-com pil ed by a VM.	Portable execution and runtime optimisation.	Good option if branches need a common virtual runtime.

Preprocessor, linker and loader

- Preprocessor: handles directives such as includes, macros or conditional compilation, if ThreatRule supports them. It is optional in a minimal language.
- Linker: combines generated object code with runtime libraries and resolves external symbols. If the output is interpreted IR, a conventional native linker may not be needed.
- Loader: places executable code or a runtime module into memory and prepares execution. In the SOC, this corresponds to loading validated rules into the rule engine.

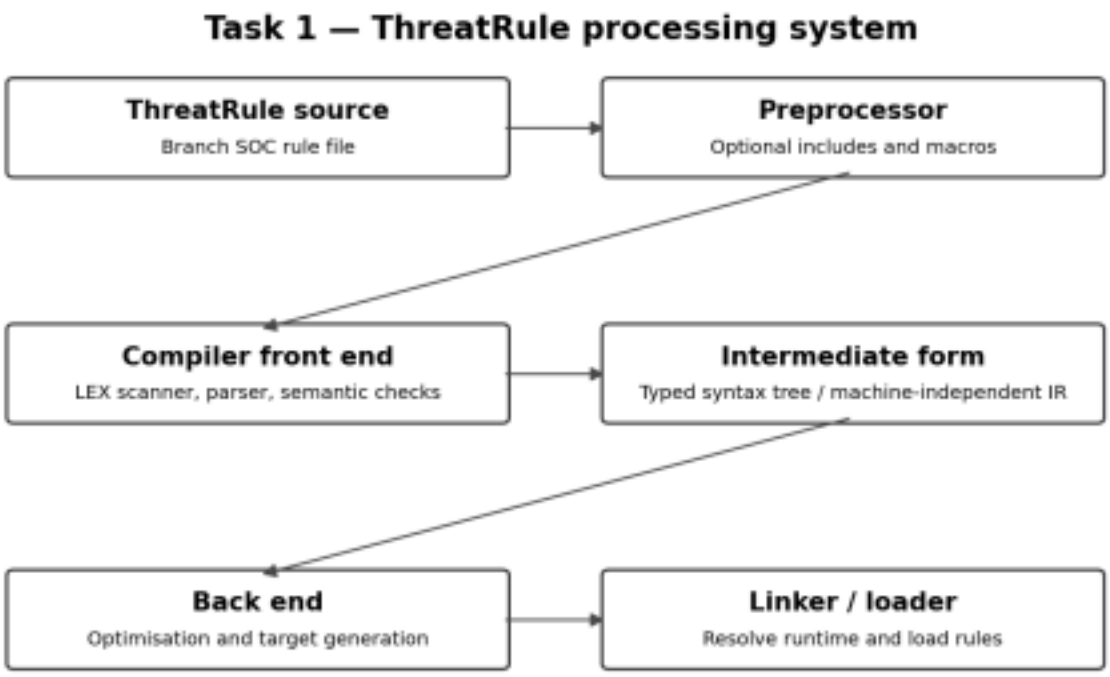


Figure 1. ThreatRule language-processing system. Linker and loader are deployment stages; an IR engine may load the compiled rules directly.

Recommended processor

Use a compiler front end with a LEX/Flex scanner and parser. Validate centrally and emit a platform-neutral IR; then interpret that IR or generate target-specific code. This keeps rule correctness independent of the security product used at each branch. An interpreter remains useful for interactive debugging, but should not replace central compile-time validation.

TASK 2 — PHASES OF A COMPILER

Sample rule:

```
RULE ssh_attack IF src_ip == 10.0.0.5 AND port == 22 AND attempts > 5 THEN
ALERT "SSH brute force" ;
```

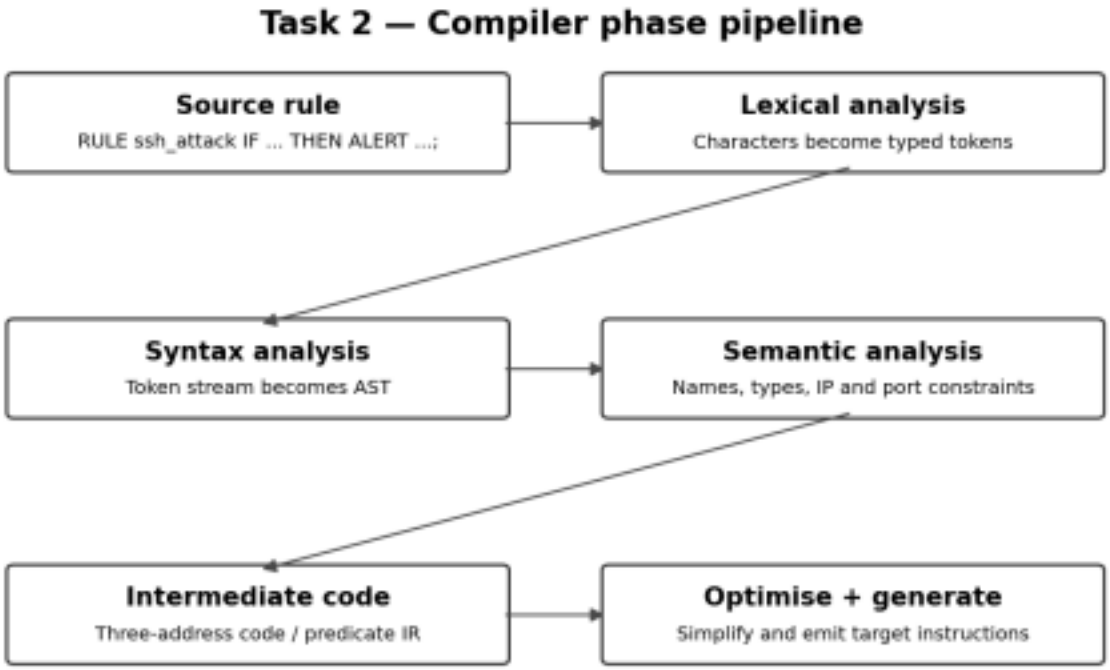


Figure 2. Each compiler phase transforms the representation and produces an inspectable output.

2.1 Lexical analysis — token stream

Lexeme	Token	Attribute
RULE	KW_RULE	—
ssh_attack	IDENT	symbol S1
IF	KW_IF	—
src_ip	IDENT	symbol S2
== 10.0.0.5 AND	EQ IP_ADDR KW_AND	— 10,0,0,5 —
port	KW_PORT / field ID	field name

==	EQ	—
22	INT	integer 22
AND	KW_AND	—
attempts	IDENT	symbol S3
>	GT	—
5	INT	integer 5
THEN	KW_THEN	—
ALERT	KW_ALERT	—
"SSH brute force"	STRING	decoded text
;	SEMICOLON	—

Whitespace is skipped and line positions are tracked. The scanner returns 22 as INT; semantic analysis checks the port range after the parser knows the field context.

2.2 Syntax analysis — abstract syntax tree

Rule\n ├── name: ssh_attack\n ├── condition: AND\n │ ├── EQ(Identifier(src_ip), IP(10,0,0,5))\n │ ├── EQ(Identifier(port), Integer(22))\n │ └── GT(Identifier(attempts), Integer(5))\n └── action: ALERT(String("SSH brute force"))

The parser verifies RULE identifier IF condition THEN action semicolon. The AST preserves operator nesting and precedence.

2.3 Semantic analysis

- Name resolution: ssh_attack is a valid rule name; src_ip, port and attempts must exist in the ThreatRule field schema.
- Type checking: src_ip is compared with an IP value; port and attempts are numeric;

AND operands are boolean; ALERT receives a string.

- Range checking: IP octets are 0–255; port is 0–65535; integer overflow is rejected.
- Policy checking: detect duplicate rule names, unsupported fields, disallowed actions and branch permission violations.

For this sample, 10.0.0.5 is a valid IPv4 address, 22 is a valid port, attempts > 5 is a numeric comparison, and the alert message is a string. These checks assume the SOC schema defines the three fields as described.

2.4 Intermediate representation — three-address code

t1 = LOAD_FIELD src_ip\nt2 = CONST_IP 10.0.0.5\nt3 = EQ t1, t2\nt4 = LOAD_FIELD port\nt5 = CONST_INT 22\nt6 = EQ t4, t5\nt7 = AND t3, t6\nt8 = LOAD_FIELD attempts\nt9 = CONST_INT 5\nt10 = GT t8, t9\nt11 = AND t7, t10\nIF t11 GOTO L_alert\nGOTO L_end\nL_alert: EMIT_ALERT "SSH brute force"\nL_end:

2.5 Code optimisation

Remove unnecessary temporaries and combine boolean tests without changing short-circuit semantics. No constant-folding of run-time fields is possible.

IF (src_ip == 10.0.0.5) AND (port == 22) AND (attempts > 5)\n EMIT_ALERT("SSH brute force")

If short-circuit evaluation is supported, predicates may be ordered for lower evaluation cost only when doing so preserves semantics and no side effects are reordered.

2.6 Target code generation

LOAD_FIELD src_ip\nCONST_IP 10.0.0.5\nEQ\nJUMP_IF_FALSE L_end\nLOAD_FIELD port\nCONST_INT 22\nEQ\nJUMP_IF_FALSE L_end\nLOAD_FIELD attempts\nCONST_INT 5\nGT\nJUMP_IF_FALSE L_end\nALERT "SSH brute force"\nL_end: HALT

This is representative SOC virtual-machine bytecode, not a claim about a specific vendor instruction set.

2.7 Symbol table and error handler

The symbol table stores identifier spelling, stable symbol ID, declared type, scope, source location and field metadata. The error handler collects lexical, syntax and semantic diagnostics with line/column, offending lexeme, category and recovery action. It should continue after recoverable errors while avoiding misleading cascaded diagnostics.

TASK 3 — GROUPING OF PHASES

The front end is source-language dependent and checks whether ThreatRule is valid. The back end consumes the intermediate representation and maps it to a particular execution platform.

Front end	**Back end**
Lexical analysis using LEX/Flex	IR optimisation
Syntax analysis / parser	Target-specific instruction selection
Semantic analysis and symbol-table checks	Register and memory allocation (native target)
AST and IR construction	Code emission and packaging

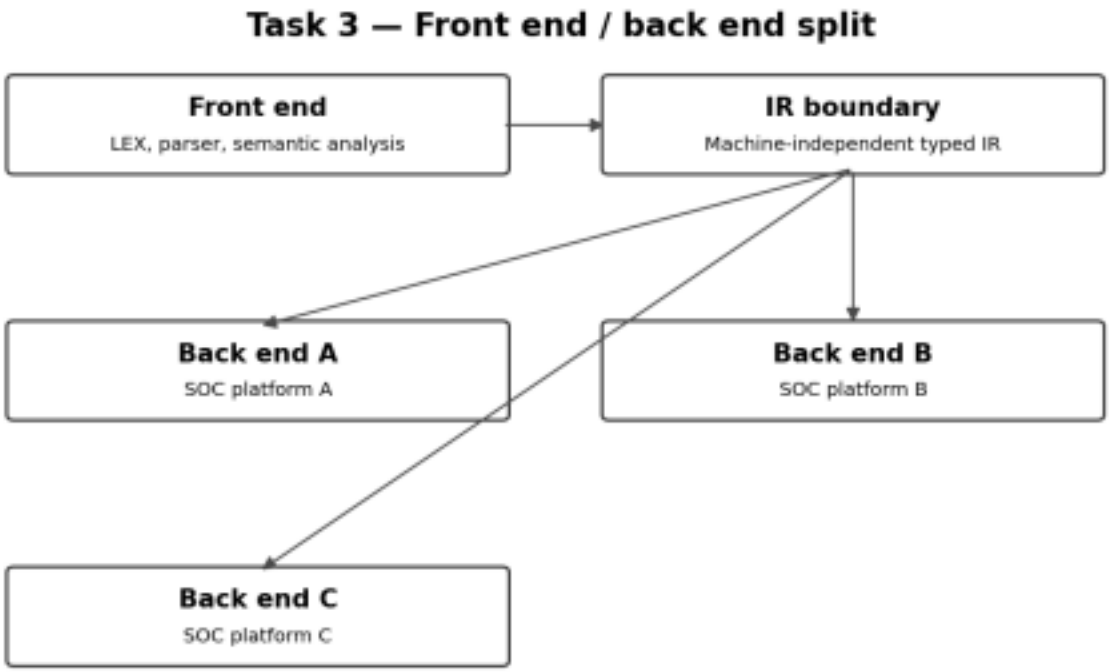


Figure 3. One common ThreatRule front end can feed several target-specific SOC back ends.

A common front end ensures that all branches use the same language rules and receive consistent diagnostics even when they use different SIEM products or endpoint tools. Only the back end or IR adapter changes for each target.

A pass is one traversal over a representation or input that performs a defined compiler

operation. A one-pass compiler attempts most work in one traversal; it may be fast but is difficult when later information is needed earlier. A multi-pass design uses explicit stages and IR. ThreatRule should use multiple passes: scan, parse, semantic analysis, IR construction, optimisation and target generation. This is easier to test, diagnose and extend.

TASK 4 — COMPILER CONSTRUCTION TOOLS

Tool category	**Purpose**	**ThreatRule placement**
Scanner generator (LEX/Flex)	Generates scanner from regular-expression rules.	First front-end phase; characters to tokens.
Parser generator (Yacc/Bison)	Generates parser from context-free grammar.	Consumes tokens and builds parse tree / AST.
Syntax-directed translation engine	Attaches semantic actions to grammar productions.	AST construction, type checks and IR creation.
Code-generator generator	Assists mapping IR / machine descriptions to target code.	Back end emits SOC bytecode or native instructions.
Data-flow analysis engine	Analyses definitions, uses and control flow.	Optimisation: liveness and safe transformations.
Compiler construction toolkit	Reusable IR, diagnostics, symbol-table and optimisation utilities.	Supports front end and back end.

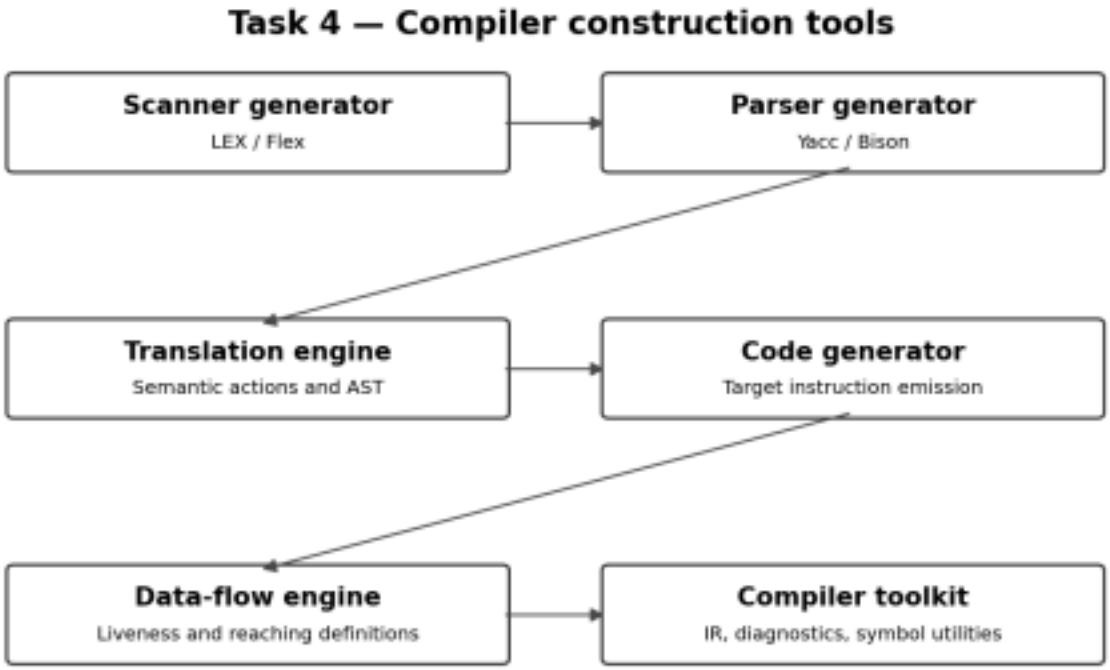


Figure 4. Compiler construction tools placed in the ThreatRule compiler pipeline.

Why LEX/Flex instead of a hand-written scanner?

- Regular-expression rules make token definitions explicit, reviewable and easier to extend.
- Generated scanners implement state-machine recognition and longest-match selection.
- Standard interfaces include yytext, yyleng, yylval and yylineno.
- Rules can be tested independently and maintained as token types evolve.

A hand-written scanner is reasonable for unusual lexical requirements or exact custom control. ThreatRule mostly uses regular token patterns, so LEX/Flex is easier to audit and maintain.

TASK 5 — ROLE OF THE LEXICAL ANALYZER

The lexical analyser is the first compiler phase. It reads characters, groups them into lexemes, recognises token classes, attaches attributes and supplies tokens to the parser. It skips whitespace and comments while tracking source positions.

Term	Meaning	ThreatRule example
Token	Abstract category returned to parser.	IDENT, IP_ADDR, EQ, KW_ALERT

Pattern	Rule describing character sequences in a token.	[A-Za-z_][A-Za-z0-9_]*
Lexeme	Actual sequence found in source.	src_ip, 10.0.0.5, "SSH brute force"

The parser calls yylex(). The scanner returns a token code and an attribute in yylval. Identifiers are interned in a symbol table and returned as symbol IDs. The parser checks grammatical structure; the scanner reports malformed character sequences.

Lexical analysis handles regular character-level patterns; syntax analysis handles nested grammatical structure. Separation simplifies both phases and allows whitespace/comments to be removed before parsing.

A buffer pair uses two input-buffer halves. The forward pointer scans characters and the lexeme-begin pointer marks the token start. Sentinels at the ends reduce bounds checks. When one half is exhausted, the other is refilled. If one character beyond a token is read, the scanner retracts or uses lookahead.

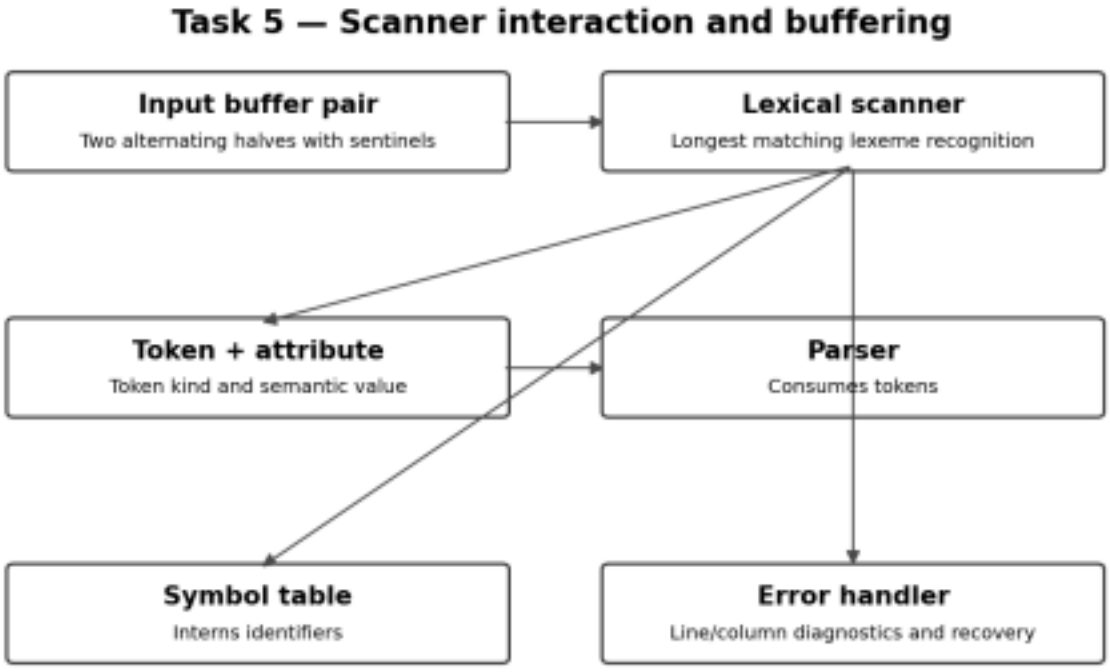


Figure 5. Scanner, parser, symbol table and diagnostic interaction; a buffer pair

supplies the input stream.

Faulty input	Error	Recovery action
src_ip == 10.0.300.5	IPv4 octet 300 exceeds 255.	Consume the complete dotted numeric candidate as one malformed IP; report line/column and resume at delimiter.
ALERT "Login failed ;	Unterminated quoted string.	Report at opening quote; discard to newline or EOF, then resume.
port == 22 @ attempts > 5	Illegal character @.	Report @, consume it, then continue scanning attempts > 5.

For the malformed IP, do not accept a partial valid address followed by unrelated tokens. For the string, this recovery assumes strings cannot cross physical lines; define this language rule explicitly.

TASK 6 — SPECIFICATION OF TOKENS

6.1 Strings, languages and operations

An alphabet Σ is a finite set of symbols. A string is a finite sequence of symbols from Σ; ε is the empty string. A language is a set of strings over Σ. Regular expressions describe regular languages. Operations include union (r|s), concatenation (rs), Kleene closure (r*), positive closure (r+) and optionality (r?).

6.2 Regular definitions

```
DIGIT [0-9]\nLETTER [A-Za-z]\nID [A-Za-z_][A-Za-z0-9_]*\nUINT
[0-9]+\nOCTET (0|[1-9][0-9]?|1[0-9][0-9]|2[0-4][0-9]|25[0-5])\nIPV4
{OCTET}\\.{OCTET}\\.{OCTET}\\.{OCTET}\nPORT
(0|[1-9][0-9]{0,3}|[1-5][0-9]{4}|6[0-4][0-9]{3}|65[0-4][0-9]{2}|655[0-2][0-9]|6553[0
-5])\nSTRING \"([^\\\"\\\\\\n]|\\\\.)*\"\nWS [ \\t\\r]+\nCOMMENT \\#[^\\n]*
```

PORT denotes 0–65535. The scanner returns numeric literals as INT to avoid classifying every integer as a port; semantic analysis applies the PORT range when the grammar identifies a port field.

6.3 Token table

Token	Pattern / recognition	Example	Attribute
Keywords	Exact reserved words, checked after ID	RULE, IF, AND, OR, THEN, ALERT, BLOCK	Token code
IDENT	{ID}, excluding reserved words	ssh_attack, src_ip	Symbol-table ID
INT	{UINT}	22, 5, 65535	Integer value
IP_ADDR	{IPV4}	10.0.0.5	Four octets / packed IP
STRING	{STRING}	"SSH brute force"	Decoded string
EQ / NE	== / !=	==, !=	None
LT / LE	< / <=	<, <=	None
GT / GE	> / >=	>, >=	None
LPAREN / RPAREN	\\(/ \\)	(,)	None
SEMICOLON	;	;	None
COMMENT	#[^\\n]* (if # comments are adopted)	# branch rule	Discard; count lines

WHITESPACE		spaces, tabs, CR/LF space / tab / newline Discard; update line	

Keyword separation: match a complete identifier, then perform exact reserved-word lookup. If it is a keyword, return the keyword token; otherwise return IDENT and install it in the symbol table. This prevents IFACE being split into IF + ACE. Explicit keyword rules should precede ID rules where equal-length matches are possible.

The brief explicitly names RULE, IF, AND, OR, THEN, ALERT and BLOCK. The sample also uses port as a field name. The grammar must consistently decide whether port is reserved or a predefined field identifier.

TASK 7 — RECOGNITION OF TOKENS

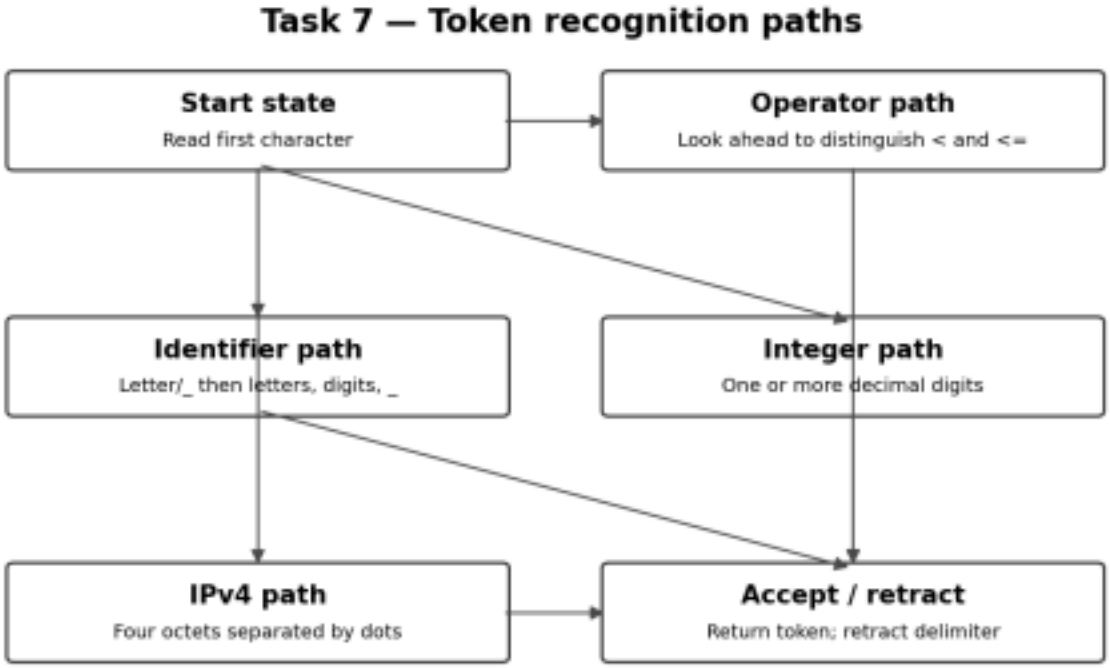


Figure 6. Recognition paths from the start state, including lookahead and accepting/retraction behaviour.

Token class	Transition description	Accept / retract

Relational operators	q0 on < → q1; q1 on = → LE, else LT. q0 on > → q2; q2 on = → GE, else GT. q0	Accept two-character operator; retract nonmatching lookahead.

	on = then = → EQ; q0 on ! then = → NE.	
Identifier / keyword	q0 on letter or _ → qID; qID loops on letters, digits, _.	On delimiter accept accumulated lexeme and retract delimiter; reserved-word lookup.
Integer	q0 on digit → qINT; qINT loops on digits.	On nondigit accept and retract; detect integer overflow.
IPv4	Read OCTET, dot, OCTET, dot, OCTET, dot, OCTET.	Accept after fourth octet; validate 0–255. Consume malformed dotted candidate as one error.

Lookahead: for <=, read < then inspect = and accept LE. For < followed by whitespace, accept LT and retract the whitespace so the next scan begins at that character.

Lexeme	**Hand trace**	**Result**
src_ip	q0 → qID on s; remain qID for r,c,_,i,p; delimiter ends lexeme.	IDENT(src_ip)

10.0.0.5	q0 → octet 10 → dot → octet 0 → dot → octet 0 → dot → octet 5.	IP_ADDR(10,0,0,5)
<=	q0 → qLT-lookahead on < → qLE on =.	LE
22	q0 → qINT on 2; loop on 2; delimiter ends token.	INT(22)

These are recogniser traces expressed as state transitions. The actual generated DFA should be tested with the Flex scanner; the supplied DOCX does not include an actual state table or scanner output.

TASK 8 — WRITE THE LEX SPECIFICATION

The following Flex example covers the core token set. It assumes token constants and YYSTYPE fields are declared in parser.tab.h by the parser generator; support functions must be implemented and linked.

```
%option noyywrap yylineno nodefault
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "parser.tab.h"
static int intern_identifier(const char *s);
static int keyword_token(const char *s);
static void lexical_error(const char *s, const char *why);
%}
DIGIT [0-9]
ID [A-Za-z_][A-Za-z0-9_]*
UINT [0-9]+
OCTET (0|[1-9][0-9]?|1[0-9][0-9]|2[0-4][0-9]|25[0-5])
IPV4 {OCTET}\.{OCTET}\.{OCTET}\.{OCTET}
QSTR \"([^\"\\\n]|\\.)*\"
WS [ \t\r]+
```

```
COMMENT \#[^\n]*
%%
"RULE" { return KW_RULE; }
"IF" { return KW_IF; }
"AND" { return KW_AND; }
"OR" { return KW_OR; }
"THEN" { return KW_THEN; }
"ALERT" { return KW_ALERT; }
"BLOCK" { return KW_BLOCK; }
"==" { return EQ; }
"!=" { return NE; }
"<=" { return LE; }
">=" { return GE; }
"<" { return LT; }
">" { return GT; }
"(" { return LPAREN; }
")" { return RPAREN; }
";" { return SEMICOLON; }
{IPV4} { yylval.ip_value = parse_ipv4(yytext); return IP_ADDR; }
{QSTR} { yylval.str = decode_string(yytext,yyleng); return STRING; }
{UINT} { yylval.ival = checked_integer(yytext); return INT; }
{ID} { int k=keyword_token(yytext); if(k) return k;
            yylval.sym_id=intern_identifier(yytext); return IDENT; }
{COMMENT} { /* discard comment */ }
{WS} { /* discard horizontal whitespace */ }
\\n { /* yylineno is maintained by Flex */ }
. { lexical_error(yytext,"illegal character"); }
%%
static void lexical_error(const char *s,const char *why) {
 fprintf(stderr,"lexical error at line %d: %s (%s)\\n",yylineno,why,s);
}
int main(void) {
 int t; while((t=yylex())!=0)
   printf("token=%d line=%d text=%s\\n",t,yylineno,yytext);
 return 0;
}
```

8.1 Support functions and variables

- keyword_token() performs exact reserved-word lookup. intern_identifier() inserts a spelling once and returns a stable symbol ID.
- parse_ipv4() converts the dotted address to a validated four-octet or packed representation. checked_integer() detects conversion overflow.
- decode_string() removes quotes and interprets only the language-defined escape sequences.
- yytext is the matched lexeme; yyleng is its length; yylval carries attributes; yylineno tracks lines. Add explicit column tracking if column diagnostics are required.

Flex selects the longest match. If two rules match the same longest length, the earlier rule wins. Put <= before < and >= before >. Keywords should precede ID rules or be handled by exact reserved-word lookup.

8.2 Generating lex.yy.c

```
bison -d parser.y\nflex threatrule.l\ngcc -Wall -Wextra parser.tab.c lex.yy.c support.c
-o threatrule
```

The exact command depends on parser and support files. The uploaded source does not include a .l file or build log, so this document does not claim these commands were executed successfully.

TASK 9 — IMPLEMENT AND TEST THE LEXICAL ANALYZER

A reproducible test builds the exact scanner from Task 8, runs it on generated rule files and compares actual tokens and diagnostics with an independently prepared expected result. The supplied DOCX states that sizes 100, 1,000, 10,000 and 100,000 rules were used, but the source code, inputs, logs and raw outputs are not included. The following is the required procedure, not a claim of completed execution.

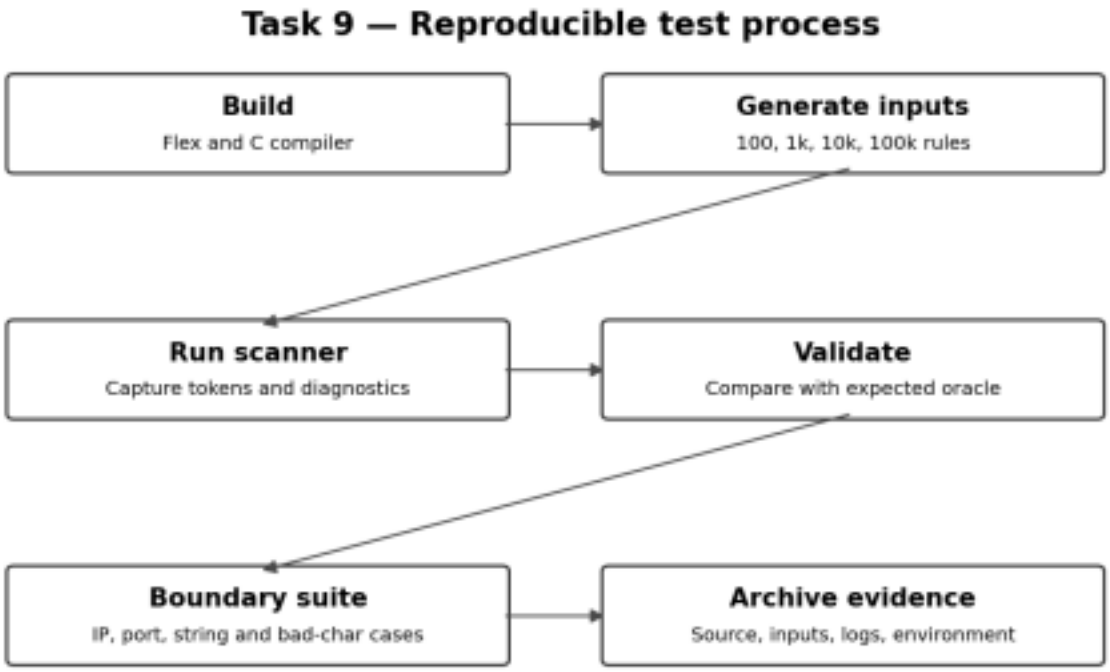


Figure 7. Reproducible scanner build, test and evidence workflow.

Test group	Input examples	Expected result
Keywords / identifiers	RULE, IF, ssh_attack, src_ip	Keywords have reserved tokens; names are IDENT.
Operators	==, !=, <, <=, >, >=	Each is one correct token; < and <= differ.
Valid IP boundaries	0.0.0.0; 255.255.255.255	IP_ADDR accepted.
Invalid IP	256.1.1.1; 10.0.300.5; 1.2.3	Malformed candidate reported once; scanning continues.
Port boundary	port 0, 22, 65535; 65536	0–65535 accepted; 65536 rejected semantically.

Strings	"SSH brute force"; unterminated quote	Valid string token; unterminated string diagnostic.
Illegal character	port == 22 @ attempts > 5	Report @ and continue with later tokens.
Comments / whitespace	Spaces, tabs, blank lines, # comment	Ignored with correct line tracking.

9.1 Scale procedure

● Record operating system, Flex version, C compiler, CPU and memory. ● Generate deterministic files containing exactly 100, 1,000, 10,000 and 100,000 rules; save the generator and random seed.

● Build once with warnings enabled and save the complete build log. ● Run a warm-up, then at least five timed scans per file size on the same machine. ● Capture rules, lines, bytes, tokens, errors, elapsed time, exit code and standard output/error.

● Compare observed tokens with an expected-token oracle; speed does not compensate for incorrect recognition.

● Archive source, generated lex.yy.c, test inputs, logs, environment and file checksums.

Acceptance criteria: valid boundaries are accepted; invalid values produce line/column diagnostics; scanner continues after recoverable errors; and valid tokens following an error are still returned.

The uploaded mark sheet awards Task 9 3.5/4 but its checklist marks successful compilation as “Partly”. No executable, source or logs are attached. Attach actual build and test evidence before claiming a completed implementation.

TASK 10 — RECORD SCANNER PERFORMANCE

Performance must be measured, not estimated. The assignment asks for rule and line counts, tokens, scanning time and lexical errors. The uploaded file refers to Table 10.1 and Figures 10–13 but does not include their numerical contents. The table below is intentionally a measurement template, not invented data.

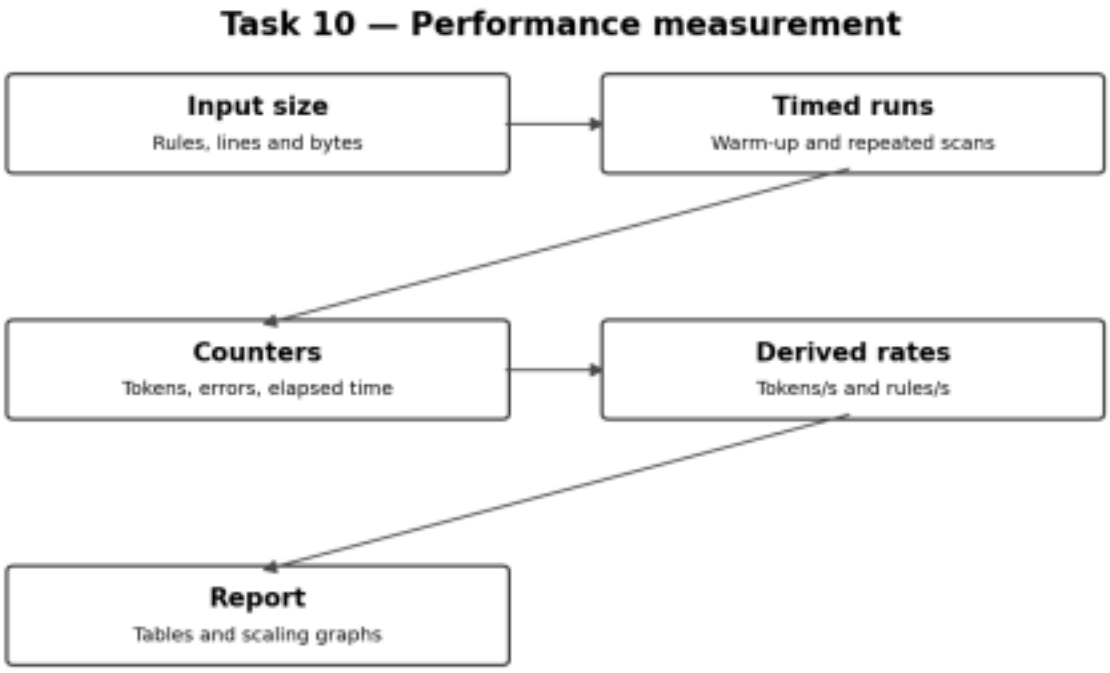


Figure 8. Performance measurement workflow.

Rules	Lines	Tokens	Errors	Time (s)	Tokens/s	Rules/s
100	measure	measure	measure	measure	calculate	calculate
1,000	measure	measure	measure	measure	calculate	calculate
10,000	measure	measure	measure	measure	calculate	calculate
100,000	measure	measure	measure	measure	calculate	calculate

Tokens/s = token count ÷ elapsed seconds. Rules/s = rule count ÷ elapsed seconds. Report median and spread across repeated runs, plus the hardware/software environment. Plot scanning time against rule count and throughput against rule count. For invalid-input runs, also plot error counts.

For regular patterns and constant-time actions, scanning should generally scale approximately linearly with input bytes. Deviations can arise from expensive symbol-table lookups, string allocation, pathological patterns, diagnostic volume or I/O. Record bytes as well as rule count because rule lengths vary.

TASK 11 — RECOMMEND AND DEFEND THE FINAL DESIGN

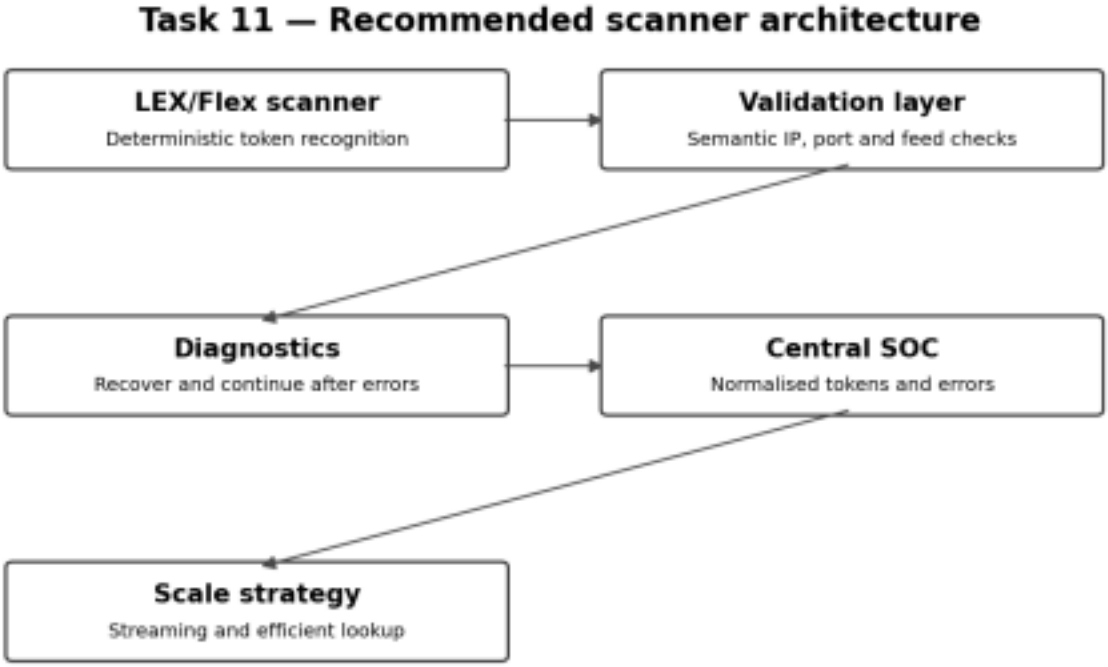


Figure 9. Recommended scanner architecture for the central SOC.

Recommendation: retain the LEX/Flex-generated scanner, but use it as one component in a layered front end. The scanner emits stable token kinds and attributes; semantic analysis validates context-dependent constraints such as port ranges and field types. Use streaming input, efficient symbol-table lookup, structured diagnostics and regression tests.

- Suitable: regular expressions express ThreatRule tokens clearly; generated scanning is appropriate for sequential large files.
- Reusable: a common front end keeps branch validation consistent. ● Limitations: lexical analysis cannot validate all context-sensitive facts; those belong in semantic analysis.
- Robustness: impose maximum token lengths for URLs and strings; recover to safe delimiters and avoid cascaded diagnostics.
- Evidence: performance claims must come from the actual scanner executable. A reference scanner mirroring the specification must not be reported as Flex-binary timing.

The mark sheet claims testing through 100,000 rules, but raw measurements are absent. The final defence should quote actual measured values once the logs and performance table are attached.

TASK 12 — RE-EVALUATE AND ADAPT THE DESIGN

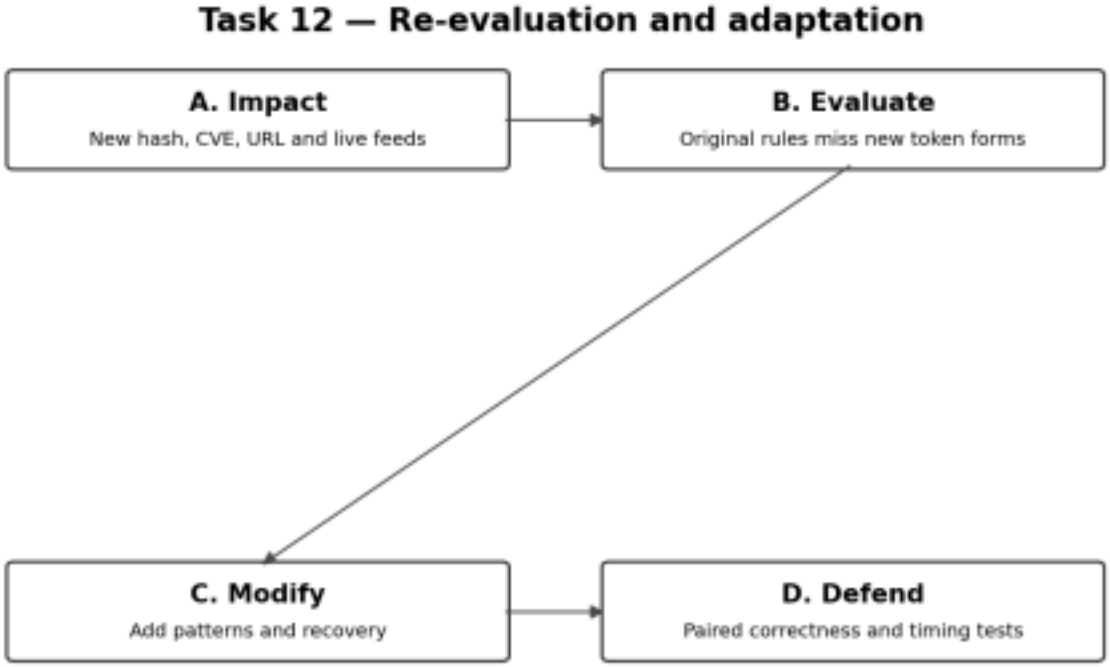


Figure 10. Impact, evaluation, modification and evidence-based defence.

12A — Impact analysis

Partner feeds introduce file hashes, CVE identifiers and URLs, and require every lexical error to be reported with a line number while scanning continues.

- Hashes: hexadecimal strings of defined lengths (for example SHA-256 has 64 hexadecimal characters). Generic identifier/integer rules may split or misclassify them.
- CVE identifiers: forms such as CVE-2024-3400 include hyphens; a generic identifier may stop at the first hyphen and integer rules may split the remainder.
- URLs: contain : / ? # & = . and percent escapes. Existing operator, comment and punctuation rules may split them unless a URL token is recognised in context. ● Continuous diagnostics: report location and offending text, consume input to make progress and resume at a safe boundary.

12B — Evaluation of original design

Without dedicated hash, CVE and URL rules, the original scanner cannot reliably return those values as single typed tokens. It may emit partial identifiers, integers and illegal-character errors. Error handling is insufficient if it terminates scanning or repeatedly reports the same malformed input.

The supplied mark sheet claims 0 of 5,000 new-token literals recognised and 13,040 spurious errors. The test corpus, expected labels, scanner output and calculation method are not included, so these figures cannot be independently verified. If verified, they strongly support modifying the original scanner.

12C — Adaptation or redesign

Decision: MODIFY. Retain the existing Flex scanner and original vocabulary, add bounded patterns for new tokens, define precedence, and improve recovery. This is lower risk than replacing the scanner because the original language remains valid.

```
HEX [0-9A-Fa-f]\nSHA256 {HEX}{64}\nCVE
CVE-[0-9]{4}-[0-9]{4,}\nSCHEME (https?|ftp)\nURLCHAR
[A-Za-z0-9._~:/?#\\[\\]@!$&()*+,;=%-]\nURL {SCHEME}://{URLCHAR}+
```

SHA256 is an example exact-length SHA-256 token. Add distinct exact-length patterns for MD5 or SHA-1 if those are required. The CVE expression captures the familiar prefix/year/sequence shape; production validation should follow current CVE naming rules. URL matching is a candidate lexical pattern; full host, percent-encoding and scheme validation belongs in a validator.

```
{URL} { yylval.str=copy_lexeme(yytext,yyleng); return URL_TOKEN; }
{CVE} { yylval.str=copy_lexeme(yytext,yyleng); return CVE_TOKEN; }
{SHA256} { yylval.str=copy_lexeme(yytext,yyleng); return HASH_TOKEN; }
/* Existing IP, string, operator, integer and identifier rules follow. */
```

Put specific token patterns before generic ID/numeric rules when equal-length matches are possible. Flex longest-match takes precedence over rule order. Define whether trailing punctuation such as a comma or closing parenthesis belongs to the URL; trim it deliberately if it does not.

- Illegal character: report line and column, consume one character, continue.
- Malformed hash/CVE/URL: consume the complete candidate to a defined delimiter, report one diagnostic, resume.
- Unterminated string: report at opening quote and recover at newline or EOF, according to the language specification.
- Guarantee progress: every error action consumes input or reaches EOF; never retry the same character indefinitely.
- Use structured diagnostics: file, line, column, category, offending text and recovery action.

12D — Evidence-based defence

Run original and modified scanners on the same fixed 5,000-literal corpus and the same original ThreatRule regression suite. Measure correct recognitions, false recognitions, spurious errors, missed errors, total diagnostics and elapsed time. The modified scanner is acceptable only if it recognises valid new tokens without breaking original rules.

Measure	**Original scanner**	**Modified scanner**	**Judgement**
New-token correct recognitions	Measure	Measure	Modified should recognise valid hash/CVE/URL set.
False positives / spurious errors	Measure	Measure	Should materially reduce errors on valid feeds.

regressionsMeasure Measure No new failures in Original-token existing vocabulary.

Elapsed time on same corpusMeasure Measure Report median and relative change.

Recovery after malformed tokenMeasure Measure Must report location and continue.

The source mark sheet reports original-design figures but does not supply modified-scanner timings or recognition counts. Therefore no numerical speed-up can honestly be claimed from this file alone. The technically justified choice is MODIFY, with quantitative defence conditional on completing and attaching the paired test.

Trade-offs: additional patterns increase complexity and may create ambiguity around URLs, comments and punctuation. Detailed recovery improves availability but can cost time on malformed feeds. Bounded token lengths and explicit validation reduce memory and denial-of-service risks.

REFERENCES

- A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., Pearson, 2006.
- J. Levine, flex & bison: Text Processing Tools, O'Reilly Media, 2009.
- Flex Project, "The Fast Lexical Analyzer," Flex manual: https://westes.github.io/flex/manual/
- GNU Bison, "Bison Manual": https://www.gnu.org/software/bison/manual/
- MITRE, "CVE Program": https://www.cve.org/
- RFC 3986, Uniform Resource Identifier (URI): Generic Syntax, IETF, 2005: https://www.rfc-editor.org/rfc/rfc3986
- RFC 791, Internet Protocol, IETF, 1981: https://www.rfc-editor.org/rfc/rfc791 ● Assignment scenario, sample rule, task descriptions, rubric and reported marks are taken from the supplied AS-1 CD.docx.

FINAL SUBMISSION EVIDENCE CHECKLIST

- Attach threatrule.l and all support/header files.
- Attach parser grammar and generated parser header if submitting the full compiler.
- Attach generated lex.yy.c and successful build log.
- Attach deterministic generator and 100 / 1,000 / 10,000 / 100,000-rule input files. ● Attach valid, invalid and boundary test logs, including IP and port limits. ● Attach measured Task 10 table and graphs from the actual scanner executable. ● Attach original-versus-modified 5,000-literal corpus, logs and timing comparison for Task 12.
- Verify source code, token definitions, test oracle and reported results are consistent.